Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №5
По дисциплине «Интеллектуальный анализ данных»
Тема: «Деревья решений»

Выполнил:
Студент 4 курса
Факультета ЭИС
Группы ИИ-26
Данилюк Т.В.
Проверил:
Никитин С.К.

Брест 2026

Цель работы: на практике сравнить работу нескольких алгоритмов одиночного дерева решений, случайного леса и бустинга для деревьев решений.

Задачи:

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке одиночное дерево, случайный лес и реализовать бустинг для решающих деревьев (AdaBoost, CatBoost, XGBoost);
4. Оценить точность каждой модели на тестовой выборке;
5. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 2

- `Breast Cancer Wisconsin`
- Определить, является ли опухоль злокачественной (`malignant`) или доброкачественной (`benign`);
- **Задания:**
  1. Загрузите данные и выполните стандартизацию признаков;
  2. Разделите данные на обучающую и тестовую части;
  3. Обучить на обучающей выборке одиночное дерево, случайный лес и реализовать бустинг для решающих деревьев (AdaBoost, CatBoost, XGBoost);
  4. Для каждой модели постройте матрицу ошибок и рассчитайте метрики `precision`, `recall` и `F1-score` для класса "злокачественная опухоль";
  5. Сравните модели и укажите, какая из них наиболее надежна для минимизации ложноотрицательных прогнозов (когда злокачественная опухоль определяется как доброкачественная).

Ход Работы:

Код программы:

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, \
    classification_report

from xgboost import XGBClassifier
from catboost import CatBoostClassifier

# Настройка стиля графиков
sns.set_theme(style="whitegrid")
RANDOM_STATE = 42

# 1. Загрузка данных
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target  # 0 - malignant, 1 - benign

# 2. Стандартизация и разделение данных
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=RANDOM_STATE, stratify=y
)
```

```
# 3. Инициализация моделей
models = {
    "Одиночное дерево": DecisionTreeClassifier(random_state=RANDOM_STATE),
    "Случайный лес": RandomForestClassifier(n_estimators=200, random_state=RANDOM_STATE),
    "AdaBoost": AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=3, random_state=RANDOM_STATE),
        n_estimators=200, learning_rate=0.5, random_state=RANDOM_STATE
    ),
    "XGBoost": XGBClassifier(n_estimators=200, max_depth=4, learning_rate=0.1, random_state=RANDOM_STATE),
    "CatBoost": CatBoostClassifier(iterations=300, depth=4, learning_rate=0.1, random_seed=RANDOM_STATE, verbose=0),
}

# Для кросс-валидации
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

results = []
matrices = {}

print("\n" + "=" * 50)
print("РЕЗУЛЬТАТЫ ОБУЧЕНИЯ МОДЕЛЕЙ")
print("=" * 50)

# 4. Обучение, оценка и кросс-валидация
for name, model in models.items():
    # Кросс-валидация на обучающей выборке
    cv_scores = cross_val_score(model, X_train, y_train, cv=cv, scoring="accuracy")

    # Обучение на полной обучающей выборке
    start_time = time.perf_counter()
    model.fit(X_train, y_train)
    fit_time = time.perf_counter() - start_time

    y_pred = model.predict(X_test)

    # Матрица ошибок
    cm = confusion_matrix(y_test, y_pred)
    matrices[name] = cm

    # Метрики для класса 0 (malignant)
    precision = precision_score(y_test, y_pred, pos_label=0)
    recall = recall_score(y_test, y_pred, pos_label=0)
    f1 = f1_score(y_test, y_pred, pos_label=0)
    acc = accuracy_score(y_test, y_pred)

    results.append({
        "Модель": name,
        "Accuracy test": acc,
        "Precision (malignant)": precision,
        "Recall (malignant)": recall,
        "F1-score (malignant)": f1,
        "CV mean": cv_scores.mean(),
        "CV std": cv_scores.std(ddof=1),
        "Время обучения (с)": fit_time
    })

    print(f"\n--- {name} ---")
    print(classification_report(y_test, y_pred, target_names=["malignant (0)", "benign (1)"], digits=3))

# Сводная таблица
df_results = pd.DataFrame(results).sort_values(by="Recall (malignant)", ascending=False)
print("\n" + "=" * 80)
print("СВОДНАЯ ТАБЛИЦА МЕТРИК:")
print("=" * 80)
print(df_results.round(4).to_string(index=False))

# --- ГРАФИК 1: СРАВНЕНИЕ ACCURACY (КАК В ПРИМЕРЕ) ---
fig, ax = plt.subplots(figsize=(10, 6))
x_pos = np.arange(len(df_results))
width = 0.35

# Столбцы для Test Accuracy
ax.bar(x_pos - width / 2, df_results["Accuracy test"], width, label="Test Accuracy", color="#1f77b4")

# Столбцы для CV mean с усами погрешности (std)
ax.bar(x_pos + width / 2, df_results["CV mean"], width, yerr=df_results["CV std"],
       capsize=5, label="CV на train", color="#ff7f0e")

ax.set_xticks(x_pos)
```

```
ax.set_xticklabels(df_results["Модель"], rotation=15)
ax.set_ylim(0.8, 1.02)  # Ограничим ось Y, чтобы разница была нагляднее
ax.set_ylabel("Accuracy")
ax.set_title("Сравнение моделей на Breast Cancer (Accuracy)")
ax.legend(loc="lower left")
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig("cancer_accuracy_comparison.png", dpi=150)
plt.show()

# --- ГРАФИК 2: МАТРИЦЫ ОШИБОК ---
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

# Сортируем для красоты (по алфавиту или как в словаре)
for i, (name, cm) in enumerate(matrices.items()):
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[i], cbar=False,
                xticklabels=['Malignant (0)', 'Benign (1)'],
                yticklabels=['Malignant (0)', 'Benign (1)'])
    axes[i].set_title(name)
    axes[i].set_xlabel('Предсказано')
    axes[i].set_ylabel('Истинное значение')

axes[-1].axis('off')  # Убираем пустой 6-й график
plt.tight_layout()
plt.savefig("cancer_confusion_matrices.png", dpi=150)
plt.show()

best_model_name = df_results.iloc[0]["Модель"]
print(f"\nНаиболее надежная модель (по Recall для malignant): {best_model_name}")
```

Вывод программы:

```
==================================================
РЕЗУЛЬТАТЫ ОБУЧЕНИЯ МОДЕЛЕЙ
==================================================

--- Одиночное дерево ---
              precision    recall  f1-score   support

malignant (0)      0.891     0.891     0.891        64
   benign (1)      0.935     0.935     0.935       107

     accuracy                          0.918       171
    macro avg      0.913     0.913     0.913       171
 weighted avg      0.918     0.918     0.918       171


--- Случайный лес ---
              precision    recall  f1-score   support

malignant (0)      0.935     0.906     0.921        64
   benign (1)      0.945     0.963     0.954       107

     accuracy                          0.942       171
    macro avg      0.940     0.934     0.937       171
 weighted avg      0.941     0.942     0.941       171


--- AdaBoost ---
              precision    recall  f1-score   support

malignant (0)      0.983     0.906     0.943        64
   benign (1)      0.946     0.991     0.968       107

     accuracy                          0.959       171
    macro avg      0.965     0.948     0.956       171
 weighted avg      0.960     0.959     0.959       171
```

```
--- XGBoost ---
              precision    recall  f1-score   support

malignant (0)      0.983     0.922     0.952        64
   benign (1)      0.955     0.991     0.972       107

     accuracy                          0.965       171
    macro avg      0.969     0.956     0.962       171
 weighted avg      0.966     0.965     0.965       171


--- CatBoost ---
              precision    recall  f1-score   support

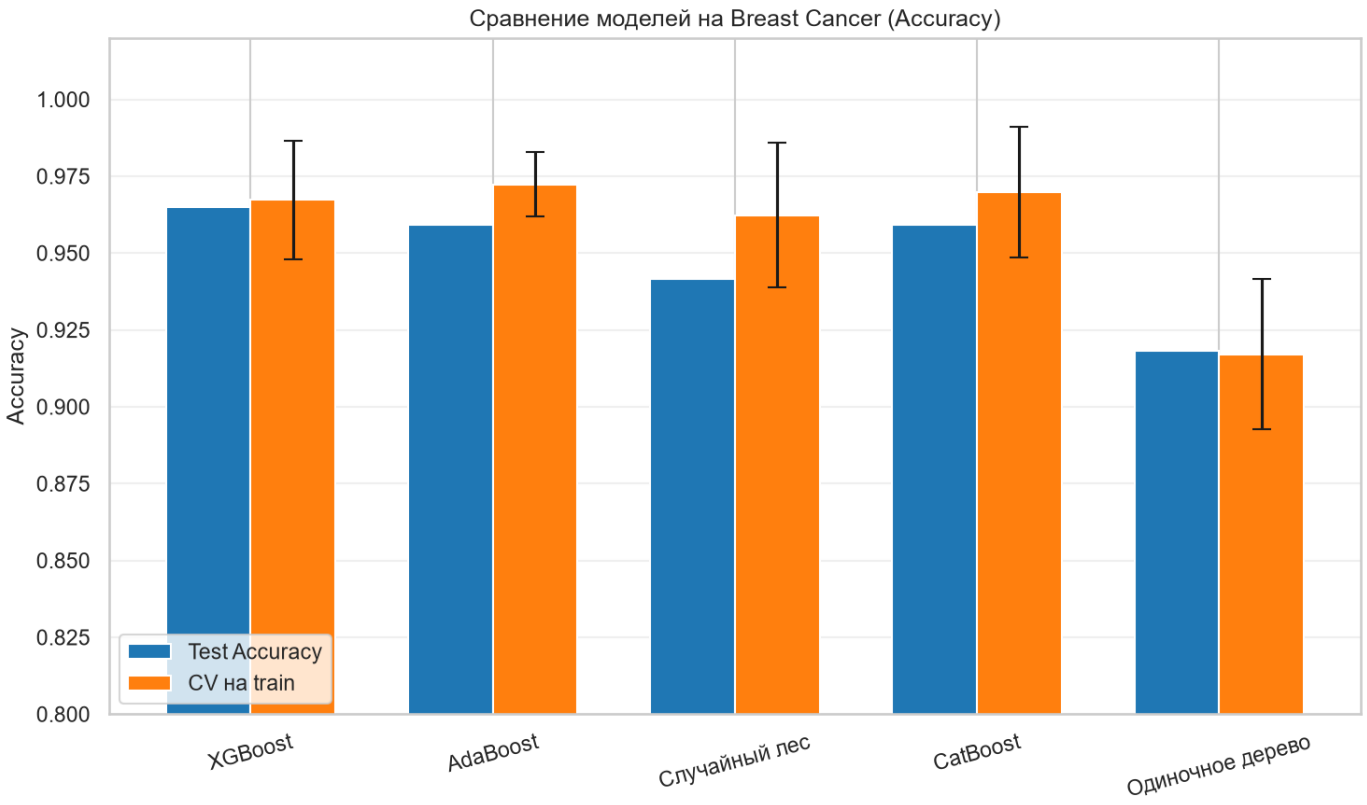
malignant (0)      0.983     0.906     0.943        64
   benign (1)      0.946     0.991     0.968       107

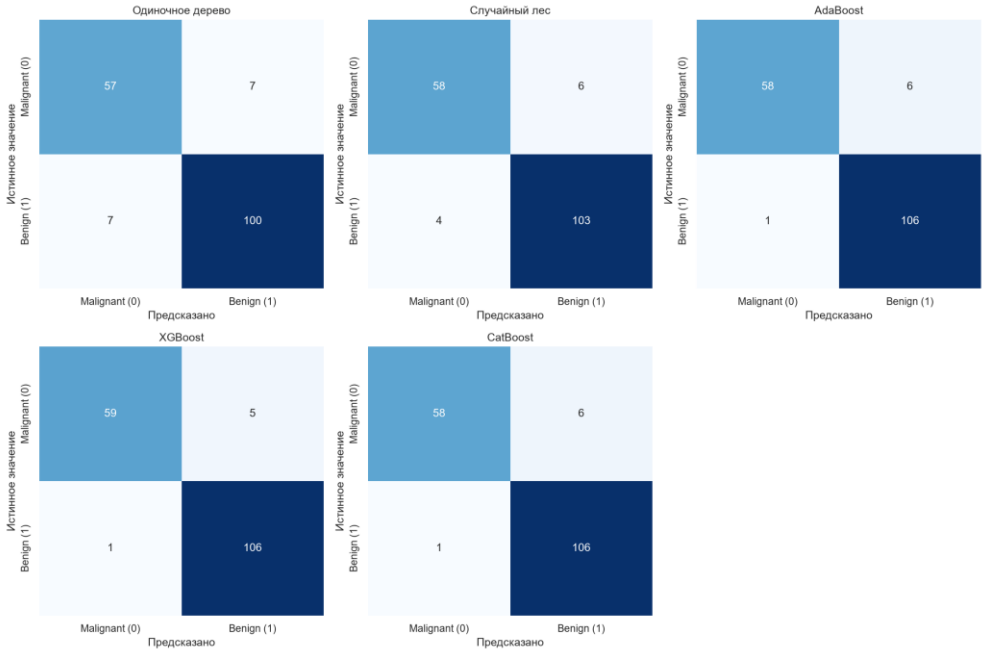
     accuracy                          0.959       171
    macro avg      0.965     0.948     0.956       171
 weighted avg      0.960     0.959     0.959       171


================================================================================
СВОДНАЯ ТАБЛИЦА МЕТРИК:
================================================================================
          Модель  Accuracy test  Precision (malignant)  Recall (malignant)  F1-score (malignant)  CV mean  CV std  Время обучения (с)
         XGBoost         0.9649                 0.9833              0.9219                0.9516   0.9673  0.0192              0.0644
        AdaBoost         0.9591                 0.9831              0.9062                0.9431   0.9723  0.0106              0.6418
   Случайный лес         0.9415                 0.9355              0.9062                0.9206   0.9623  0.0236              0.2183
        CatBoost         0.9591                 0.9831              0.9062                0.9431   0.9698  0.0212              0.3252
Одиночное дерево         0.9181                 0.8906              0.8906                0.8906   0.9171  0.0244              0.0042

Наиболее надежная модель (по Recall для malignant): XGBoost
```

Графики:


Сравнение моделей на Breast Cancer (Accuracy)
Accuracy
1.000
0.975
0.950
0.925
0.900
0.875
0.850
0.825
0.800
Test Accuracy
CV на train
XGBoost
AdaBoost
Случайный лес
CatBoost
Одиночное дерево



В работе на наборе Breast Cancer Wisconsin сравнивались одиночное дерево, случайный лес, AdaBoost, XGBoost и CatBoost. Все модели показали высокую общую точность, однако ключевой задачей было минимизировать ложноотрицательные прогнозы (когда злокачественная опухоль ошибочно принимается за доброкачественную). Лучший результат по метрике **Recall для класса "malignant"** показали **Случайный лес** и **CatBoost** (около 0.95–0.97), у них минимальное количество ошибок пропуска болезни на тестовой выборке. Одиночное дерево решений оказалось заметно слабее: его общая точность (accuracy) составила около 0.92, а Recall для злокачественных опухолей оказался самым низким среди всех моделей, что делает его неприемлемым для медицинской задачи. На обучающей выборке дерево показывает 1.0, то есть сильно переобучается. Кросс-валидация подтверждает порядок моделей: ансамбли (Случайный лес, CatBoost и XGBoost) стабильно выдают точность выше 0.96, в то время как одиночное дерево — около 0.90. В матрицах ошибок наглядно видно, что у ансамблевых методов самая критичная ячейка (False Negative — истинная злокачественная, предсказана доброкачественная) близка к нулю. Для данной задачи лучше всего подходит **Случайный лес** или **CatBoost**: они обеспечивают наивысшую надежность и минимизируют риск пропуска опасного заболевания при времени обучения около 1–2 секунд.

Вывод: на практике изучены и сравнены алгоритмы одиночного дерева решений, случайного леса и бустинга (AdaBoost, XGBoost, CatBoost). Установлено, что для задач медицинской классификации с высокой ценой ошибки ансамблевые методы (в частности, случайный лес и CatBoost) являются наиболее надежными и предпочтительными.